Feature Extraction in NLP

Feature extraction Method

- Bag-of-words
- TF-IDF

Bag-of-words

- Feature extraction method from text for use in modeling such as with machine learning algorithms.
- A bag-of-words is a representation of text that describes the occurrence of words within a document.
- It involves two things
  - A vocabulary of known words
  - A measure of the presence of known words

Example of the Bag-of-Words model

- First few lines of text from the book "A tale of Two cities" by Charles Dickens

doc1 ← • It was the best of times,
doc2 ← • it was the worst of times,
doc3 ← • it was the age of wisdom,
doc4 ← • it was the age of foolishness

} Data collection step1

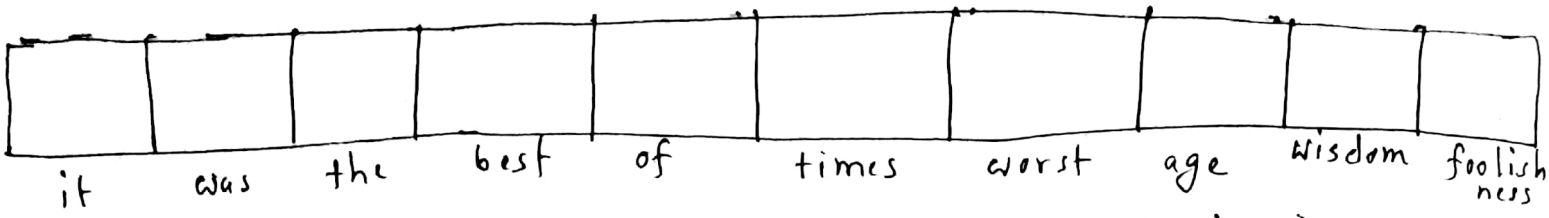
- Let's treat each line as a separate "document" and the 4 lines as our entire corpus of documents
- Make a list of all of the words in our model vocabulary.

step2 Design vocabulary

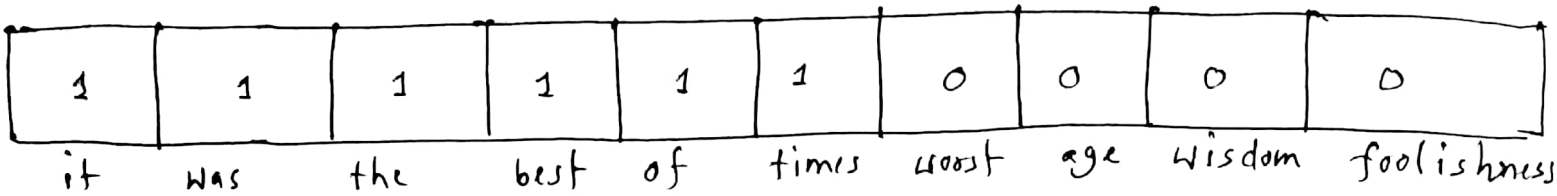
- The unique words here (ignoring case and punctuation) are:
  - it • was • the • best • of • times • worst • age • wisdom • foolishness → 10 words from a corpus containing 24 words

Scanned with CamScanner

- step3: To score the words in each document
(To mark the presence of words as a boolean value, 0 for absent, 1 for present)



Now scoring for document-1 (it was the best of times)



Vector will be [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]

↓

Machine Learning tool
- Neural Network
- Support vector Machine
- Decision Tree
- Nearest Neighbor
- Hidden Markov Model
- k-Nearest Neighbor
- Logistic Regression
- ⋮

Scoring Words

counts → Count the number of times each words in a document

Frequency → each word appears in a document out of all the words in the document

Scanned with CamScanner

Managing Vocabulary

- To reduce the vector size

Text cleaning Technique in first step

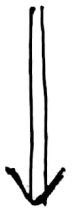
- Ignoring case
- Ignoring punctuation
- Ignore stop words like "a", 'of", etc
- Fixing misspelled words
- Reducing words to their stem (play from "playing") using stemming algorithms.

Modified Bag-of-Words Method

- To create a vocabulary of grouped words (Bigram, Trigram)
- Only the bigrams that appear in the corpus are modeled, not all possible bigrams

For example,

"It was the best of times"



Bigram generation

it was, was the, the best, best of, of times

Word Hashing (Hash trick/ feature hashing)

CAT, DOG

Draw a Hash Table using the given two words where table size is 4.

Scanned with CamScanner

Words	Bit Representation	Hash Index Generation
CAT	00010 00000 10011 = 2067	2067 mod 4 = 3
DOG	00011 01110 00110 = 3526	3526 mod 4 = 2

A → 00000 → 0
B → 00001 → 1
C → 00010 → 2
D → 00011 → 3
⋮
Z → 11001 → 25

English. Total 26 characters
Total encoded bit
$= \lceil \log_2 26 \rceil$
= 5 bits

CAT bit stream: 00010 00000 10011

$= 0*2^{14} + 0*2^{13} + 0*2^{12} + \underline{1*2^{11}} + 0*2^{10}$
$+ \cdots + \underline{1*2^{4}} + 0*2^{3} + 0*2^{2} + \underline{1*2^{1}} + \underline{1*2^{0}}$

$= 2048 + 16 + 2 + 1$

$= 2067$

Hash Table

	0
	1
DOG	2
CAT	3

Vector

0	0
0	1
1	2
1	3

DOG bit stream

$= 1*2^{11} + 1*2^{10} + 1*2^{8} + 1*2^{7} + 1*2^{6} + 1*2^{2} + 1*2^{1}$

$= 2048 + 1024 + 256 + 128 + 64 + 4 + 2$

$= 3526$

Scanned with CamScanner

TF-IDF

TF ⟶ Term frequency

IDF ⟶ Inverse Document Frequency -

TF: Term Frequency

- Frequency of the word in the current document

IDF: Inverse Document Frequency

- How reare the word is across documents

Formula

$$tf(t,d) = \frac{f_{t,d}}{\text{number of words in } d}$$

$$idf(t,D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

$$tf\text{-}idf(t,d,D) = tf(t,d) * idf(t,D)$$

Example 1 considering word "this"

Document 1

Term	Term count
this	1
is	1
a	2
sample	1

Document 2

Term	Term Count
this	1
is	1
another	2
example	3

Scanned with CamScanner

$$tf("this", d_1) = \frac{1}{1+1+2+1} = \frac{1}{5} = 0.2$$

$$tf("this", d_2) = \frac{1}{1+1+2+3} = \frac{1}{7} = 0.14$$

$$idf("this", D) = \log\left(\frac{\text{Total no. of documents}}{\text{Total no. of documents that contain term}}\right)$$

$$= \log_{10}\left(\frac{Doc1 + Doc2}{Doc1 + Doc2}\right)$$

$$= \log_{10}\left(\frac{2}{2}\right) = 0$$

So, $tf\text{-}idf("this", d_1, D) = 0.2 * 0 = 0$

$tf\text{-}idf("this", d_2, D) = 0.14 * 0 = 0$

Since tf-idf is zero for the word "this", which implies that the word is not very informative as it appears in all documents.

Example-2 : considering word "example"

$$tf("example", d_1) = \frac{0}{5} = 0$$

$$tf("example", d_2) = \frac{3}{7} = 0.429$$

$$idf("example", D) = \log_{10}\left(\frac{Doc1 + Doc2}{Doc2}\right) = \log_{10}\left(\frac{2}{1}\right) = 0.301$$

Finally,

$$tf\text{-}idf("example", d_1, D) = tf("example", d_1) * idf("example", D)$$
$$= 0 * 0.301 = 0$$

$$tf\text{-}idf("example", d_2, D) = tf("example", d_2) * idf("example", D)$$
$$= 0.429 * 0.301 \approx 0.129$$

Scanned with CamScanner